

ПЕТРОЗАВОДСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНСТИТУТ МАТЕМАТИКИ И ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
КАФЕДРА ИНФОРМАТИКИ И МАТЕМАТИЧЕСКОГО ОБЕСПЕЧЕНИЯ

Направление подготовки магистратуры
01.04.02 - Прикладная математика и информатика

Отчет о выполнении проекта

КЛАССИФИКАЦИЯ СЛОВ ПО ЧАСТЯМ РЕЧИ НА ОСНОВЕ
СИМВОЛЬНЫХ И МОРФОЛОГИЧЕСКИХ ПРИЗНАКОВ

Выполнили:

студенты 5 курса группы 22503

О. А. Плугин _____
подпись

Л. М. Чащин _____
подпись

Преподаватель:

А. С. Румянцев, д.ф.-м.н., профессор

подпись

Содержание

1	Предметная область	3
2	Прикладная задача	3
3	Полученное решение	4
3.1	Процесс обучения моделей	4
3.2	Описание обученных моделей	5
3.3	Оценка качества классификации	5
3.4	Анализ матриц ошибок	6
3.5	Выбор итоговой модели	7
A	Лог обучения моделей	7

1 Предметная область

Предметной областью данного проекта является автоматическая обработка естественного языка (Natural Language Processing, NLP), а именно задача классификации слов по частям речи (Part-of-Speech Tagging).

В рамках проекта рассматривается задача определения грамматической категории отдельного слова (существительное, глагол или прилагательное) исключительно на основе его орфографической формы, без учёта контекста предложения. Такая постановка задачи относится к подзадачам статистической обработки текстовых данных и морфологического анализа и может рассматриваться как упрощённый вариант классической задачи POS-теггинга.

В качестве исходных данных используется размеченный лингвистический корпус Universal Dependencies, содержащий слова и соответствующие им универсальные теги частей речи (UPOS). Для обработки данных и построения моделей применяются программные инструменты анализа данных и машинного обучения на языке Python.

В процессе решения задачи используются методы извлечения признаков из текстовых данных, включая символьные n-граммы и простые морфологические характеристики слов (длина слова, количество гласных, наличие характерных суффиксов). На основе полученных признаков обучаются и сравниваются две модели машинного обучения: линейный метод опорных векторов (Linear Support Vector Machine) и наивный байесовский классификатор (Multinomial Naive Bayes).

Качество работы моделей оценивается с использованием стандартных метрик классификации, а результаты представлены в виде отчётов и матриц ошибок, что позволяет провести сравнительный анализ эффективности применяемых подходов.

2 Прикладная задача

Прикладная задача проекта заключается в разработке программного решения для автоматического определения части речи отдельного слова по его написанию с использованием методов машинного обучения и программных инструментов обработки данных.

Данная задача является актуальной для прикладных задач обработки текстовой информации, таких как предварительный морфологический анализ, автоматическая аннотация корпусов, обработка пользовательского ввода, а также построение вспомо-

гательных лингвистических инструментов. В ряде практических сценариев требуется определить грамматическую категорию слова при отсутствии контекста, например при анализе словарей, поисковых запросов или отдельных токенов.

В рамках проекта требуется:

- загрузить и подготовить размеченные текстовые данные из открытого лингвистического корпуса;
- выполнить очистку и фильтрацию данных для выделения целевых классов;
- преобразовать текстовые данные в числовое представление, пригодное для машинного обучения;
- обучить модели классификации и подобрать их гиперпараметры;
- оценить качество полученных моделей с использованием метрик классификации;
- выполнить сравнительный анализ различных подходов к решению задачи.

Ограничением прикладной задачи является отсутствие контекстной информации: классификация производится исключительно на основе орфографической формы слова. Это позволяет сосредоточиться на анализе возможностей символьных и морфологических признаков и на сравнении эффективности различных алгоритмов машинного обучения при решении данной задачи.

3 Полученное решение

В результате выполнения проекта было разработано программное решение для классификации слов по частям речи на основе их орфографической формы. В данном разделе описываются процесс обучения моделей, результаты оценки качества и анализ полученных ошибок классификации.

3.1 Процесс обучения моделей

Обучение моделей выполнялось с использованием обучающей выборки, сформированной на основе корпуса Universal Dependencies. Для повышения надёжности и устойчивости результатов применялась перекрёстная проверка с подбором гиперпараметров методом GridSearchCV.

В ходе обучения для каждой модели формировался лог, содержащий информацию о перебираемых комбинациях гиперпараметров, значениях метрик качества на этапах валидации, а также о выбранных оптимальных настройках. Анализ данных логов позволяет оценить стабильность процесса обучения и корректность выбора гиперпараметров.

Training model: linear_svm

Fitting 3 folds for each of 16 candidates

Best params: {'clf__C': 2, 'features__ngram__min_df': 2, 'features__ngram__ngram_range': (2, 4)}

Best CV score: 0.9626

Test macro F1: 0.9485

Test micro F1: 0.9666

Для модели Multinomial Naive Bayes наилучшее значение макро F1-меры составило 0.8782, что ниже показателей модели Linear SVM.

3.2 Описание обученных моделей

В рамках проекта были обучены и сравнены две модели машинного обучения:

- линейный метод опорных векторов (Linear Support Vector Machine), использующий комбинацию символьных n-грамм и морфологических признаков слов;
- наивный байесовский классификатор (Multinomial Naive Bayes), использующий символьные n-граммы в качестве признаков.

Для каждой модели были подобраны оптимальные значения гиперпараметров, обеспечивающие наилучшие показатели качества классификации по результатам перекрёстной проверки.

3.3 Оценка качества классификации

Оценка качества обученных моделей проводилась на тестовой выборке, не участвовавшей в процессе обучения. В качестве основных метрик использовались макро F1-мера, микро F1-мера, а также показатели точности (precision) и полноты (recall) для каждого класса.

Использование нескольких метрик позволило объективно сравнить качество классификации и оценить влияние дисбаланса классов на результаты работы моделей.

3.4 Анализ матриц ошибок

Для наглядного анализа результатов классификации были построены матрицы ошибок для каждой из обученных моделей, позволяющие определить характер и распределение ошибок классификации.

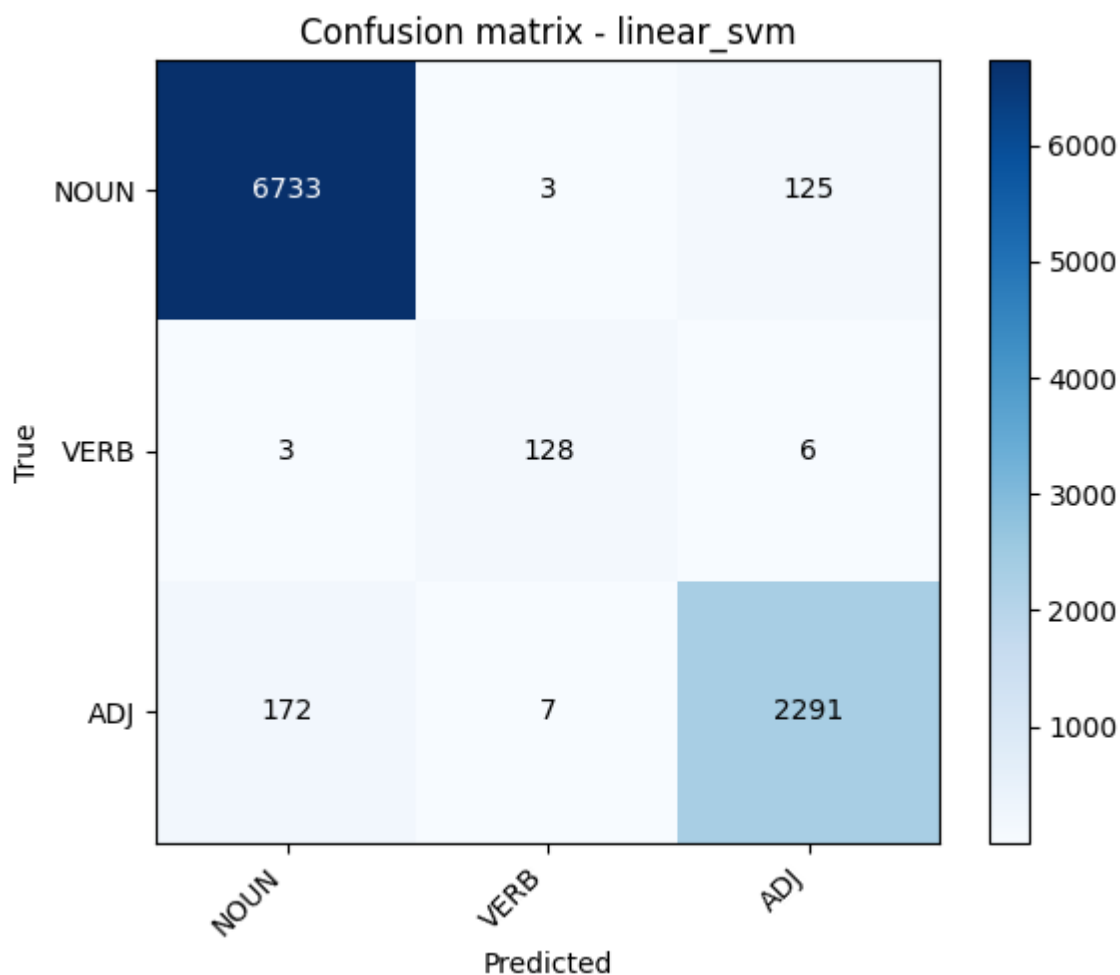


Рис. 1: Матрица ошибок для модели Linear SVM

Анализ матриц ошибок показывает, что наибольшее количество ошибок возникает при различении существительных и прилагательных, что может быть связано со схожестью их орфографических и морфологических признаков. При этом модель на основе линейного метода опорных векторов демонстрирует более высокую точность классификации по всем классам по сравнению с наивным байесовским классификатором.

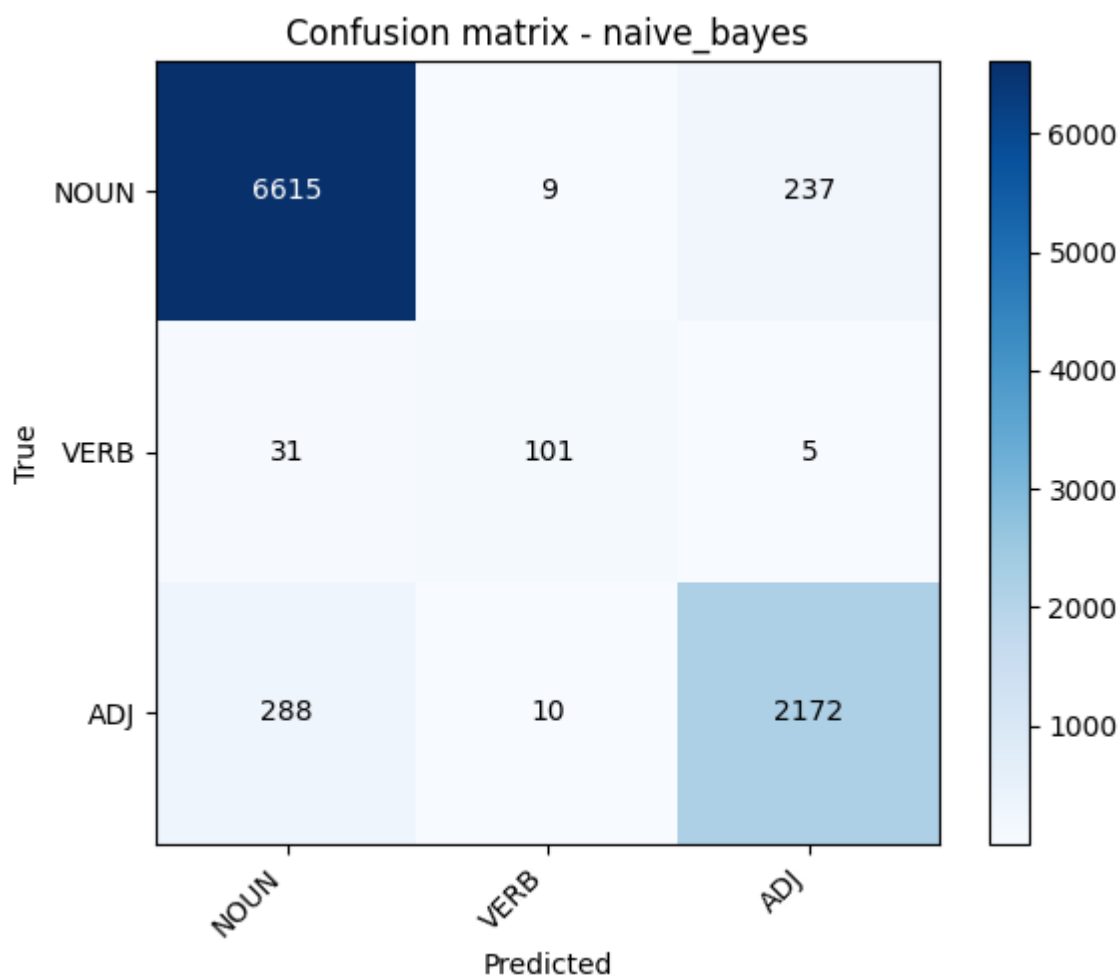


Рис. 2: Матрица ошибок для модели Naive Bayes

3.5 Выбор итоговой модели

На основании сравнительного анализа значений масго F1-меры была выбрана итоговая модель, показавшая наилучшие результаты качества классификации. Выбранная модель может быть использована повторно без необходимости повторного обучения.

А Лог обучения моделей

Training model: linear_svm

Fitting 3 folds for each of 16 candidates, totalling 48 fits

[CV] END clf__C=0.5, features__ngram__min_df=2, features__ngram__ngram_range=(2, 4); total time= 9.6s

[CV] END clf__C=0.5, features__ngram__min_df=5, features__ngram__ngram_range=(2, 4); total time= 11.7s

[CV] END clf__C=0.5, features__ngram__min_df=5, features__ngram__ngram_range=(3, 5); total time= 12.7s

[CV] END clf__C=0.5, features__ngram__min_df=2, features__ngram__ngram_range=(2, 4); total time= 13.6s

[CV] END clf__C=0.5, features__ngram__min_df=2, features__ngram__ngram_range=(2, 4); total time= 13.7s

[CV] END clf__C=0.5, features__ngram__min_df=5, features__ngram__ngram_range=(3, 5); total time= 14.1s

[CV] END clf__C=0.5, features__ngram__min_df=2, features__ngram__ngram_range=(3, 5); total time= 14.4s

[CV] END clf__C=0.5, features__ngram__min_df=2, features__ngram__ngram_range=(3, 5); total time= 14.5s

[CV] END clf__C=0.5, features__ngram__min_df=5, features__ngram__ngram_range=(3, 5); total time= 14.4s

[CV] END clf__C=0.5, features__ngram__min_df=2, features__ngram__ngram_range=(3, 5); total time= 16.1s

[CV] END clf__C=0.5, features__ngram__min_df=5, features__ngram__ngram_range=(2, 4); total time= 18.1s

[CV] END clf__C=0.5, features__ngram__min_df=5, features__ngram__ngram_range=(2, 4); total time= 18.3s

[CV] END clf__C=1, features__ngram__min_df=2, features__ngram__ngram_range=(2, 4); total time= 15.2s

[CV] END clf__C=1, features__ngram__min_df=2, features__ngram__ngram_range=(3, 5); total time= 16.0s

[CV] END clf__C=1, features__ngram__min_df=5, features__ngram__ngram_range=(3, 5); total time= 12.9s

[CV] END clf__C=1, features__ngram__min_df=5, features__ngram__ngram_range=(2, 4); total time= 18.8s

[CV] END clf__C=1, features__ngram__min_df=2, features__ngram__ngram_range=(2, 4); total time= 21.6s

[CV] END clf__C=1, features__ngram__min_df=5, features__ngram__ngram_range=(3, 5); total time= 17.7s

[CV] END clf__C=1, features__ngram__min_df=2, features__ngram__ngram_range=(3, 5); total time= 20.5s

[CV] END clf__C=1, features__ngram__min_df=5, features__ngram__ngram_range=
 =(2, 4); total time= 19.9s

[CV] END clf__C=1, features__ngram__min_df=2, features__ngram__ngram_range=
 =(2, 4); total time= 23.1s

[CV] END clf__C=1, features__ngram__min_df=5, features__ngram__ngram_range=
 =(2, 4); total time= 22.3s

[CV] END clf__C=1, features__ngram__min_df=5, features__ngram__ngram_range=
 =(3, 5); total time= 18.8s

[CV] END clf__C=1, features__ngram__min_df=2, features__ngram__ngram_range=
 =(3, 5); total time= 24.0s

[CV] END clf__C=2, features__ngram__min_df=2, features__ngram__ngram_range=
 =(2, 4); total time= 22.5s

[CV] END clf__C=2, features__ngram__min_df=2, features__ngram__ngram_range=
 =(2, 4); total time= 24.9s

[CV] END clf__C=2, features__ngram__min_df=2, features__ngram__ngram_range=
 =(3, 5); total time= 22.2s

[CV] END clf__C=2, features__ngram__min_df=2, features__ngram__ngram_range=
 =(2, 4); total time= 25.9s

[CV] END clf__C=2, features__ngram__min_df=5, features__ngram__ngram_range=
 =(2, 4); total time= 25.5s

[CV] END clf__C=2, features__ngram__min_df=5, features__ngram__ngram_range=
 =(2, 4); total time= 25.1s

[CV] END clf__C=2, features__ngram__min_df=5, features__ngram__ngram_range=
 =(3, 5); total time= 25.3s

[CV] END clf__C=2, features__ngram__min_df=2, features__ngram__ngram_range=
 =(3, 5); total time= 29.5s

[CV] END clf__C=2, features__ngram__min_df=5, features__ngram__ngram_range=
 =(2, 4); total time= 27.4s

[CV] END clf__C=2, features__ngram__min_df=2, features__ngram__ngram_range=
 =(3, 5); total time= 30.0s

[CV] END clf__C=2, features__ngram__min_df=5, features__ngram__ngram_range=
 =(3, 5); total time= 27.1s

[CV] END clf__C=2, features__ngram__min_df=5, features__ngram__ngram_range=
 =(3, 5); total time= 27.8s

```

[CV] END clf__C=5, features__ngram__min_df=2, features__ngram__ngram_range=
=(2, 4); total time= 36.0s
[CV] END clf__C=5, features__ngram__min_df=2, features__ngram__ngram_range=
=(3, 5); total time= 25.1s
[CV] END clf__C=5, features__ngram__min_df=2, features__ngram__ngram_range=
=(2, 4); total time= 32.9s
[CV] END clf__C=5, features__ngram__min_df=5, features__ngram__ngram_range=
=(3, 5); total time= 29.4s
[CV] END clf__C=5, features__ngram__min_df=5, features__ngram__ngram_range=
=(3, 5); total time= 28.6s
[CV] END clf__C=5, features__ngram__min_df=2, features__ngram__ngram_range=
=(3, 5); total time= 33.7s
[CV] END clf__C=5, features__ngram__min_df=5, features__ngram__ngram_range=
=(2, 4); total time= 31.9s
[CV] END clf__C=5, features__ngram__min_df=2, features__ngram__ngram_range=
=(3, 5); total time= 37.9s
[CV] END clf__C=5, features__ngram__min_df=5, features__ngram__ngram_range=
=(3, 5); total time= 31.9s
[CV] END clf__C=5, features__ngram__min_df=2, features__ngram__ngram_range=
=(2, 4); total time= 40.3s
[CV] END clf__C=5, features__ngram__min_df=5, features__ngram__ngram_range=
=(2, 4); total time= 33.9s
[CV] END clf__C=5, features__ngram__min_df=5, features__ngram__ngram_range=
=(2, 4); total time= 33.6s
Best params: {'clf__C': 2, 'features__ngram__min_df': 2,
'features__ngram__ngram_range=': (2, 4)}

```

Best CV score: 0.9625805352153068

Test macro F1: 0.9485

Test micro F1: 0.9666

	precision	recall	f1-score	support
ADJ	0.9459	0.9275	0.9366	2470
NOUN	0.9747	0.9813	0.9780	6861

VERB	0.9275	0.9343	0.9309	137
accuracy			0.9666	9468
macro avg	0.9494	0.9477	0.9485	9468
weighted avg	0.9665	0.9666	0.9665	9468

Training model: naive_bayes

Fitting 3 folds for each of 12 candidates, totalling 36 fits

[CV] END clf__alpha=0.1, ngram__min_df=5, ngram__ngram_range=
=(2, 4); total time= 0.4s

[CV] END clf__alpha=0.1, ngram__min_df=2, ngram__ngram_range=
=(2, 4); total time= 0.4s

[CV] END clf__alpha=0.1, ngram__min_df=2, ngram__ngram_range=
=(3, 5); total time= 0.4s

[CV] END clf__alpha=0.1, ngram__min_df=2, ngram__ngram_range=
=(3, 5); total time= 0.4s

[CV] END clf__alpha=0.1, ngram__min_df=2, ngram__ngram_range=
=(2, 4); total time= 0.4s

[CV] END clf__alpha=0.1, ngram__min_df=5, ngram__ngram_range=
=(3, 5); total time= 0.4s

[CV] END clf__alpha=0.1, ngram__min_df=5, ngram__ngram_range=
=(3, 5); total time= 0.4s

[CV] END clf__alpha=0.1, ngram__min_df=2, ngram__ngram_range=
=(3, 5); total time= 0.5s

[CV] END clf__alpha=0.1, ngram__min_df=5, ngram__ngram_range=
=(3, 5); total time= 0.4s

[CV] END clf__alpha=0.1, ngram__min_df=5, ngram__ngram_range=
=(2, 4); total time= 0.4s

[CV] END clf__alpha=0.1, ngram__min_df=5, ngram__ngram_range=
=(2, 4); total time= 0.4s

[CV] END clf__alpha=0.1, ngram__min_df=2, ngram__ngram_range=
=(2, 4); total time= 0.5s

[CV] END clf__alpha=0.5, ngram__min_df=2, ngram__ngram_range=

```

=(2, 4); total time= 0.4s
[CV] END clf__alpha=0.5, ngram__min_df=2, ngram__ngram_range=
=(2, 4); total time= 0.4s
[CV] END clf__alpha=0.5, ngram__min_df=2, ngram__ngram_range=
=(3, 5); total time= 0.4s
[CV] END clf__alpha=0.5, ngram__min_df=2, ngram__ngram_range=
=(3, 5); total time= 0.4s
[CV] END clf__alpha=0.5, ngram__min_df=2, ngram__ngram_range=
=(3, 5); total time= 0.4s
[CV] END clf__alpha=0.5, ngram__min_df=5, ngram__ngram_range=
=(3, 5); total time= 0.4s
[CV] END clf__alpha=0.5, ngram__min_df=5, ngram__ngram_range=
=(3, 5); total time= 0.4s
[CV] END clf__alpha=0.5, ngram__min_df=5, ngram__ngram_range=
=(3, 5); total time= 0.4s
[CV] END clf__alpha=0.5, ngram__min_df=5, ngram__ngram_range=
=(2, 4); total time= 0.4s
[CV] END clf__alpha=0.5, ngram__min_df=5, ngram__ngram_range=
=(2, 4); total time= 0.4s
[CV] END clf__alpha=0.5, ngram__min_df=5, ngram__ngram_range=
=(2, 4); total time= 0.4s
[CV] END clf__alpha=0.5, ngram__min_df=2, ngram__ngram_range=
=(2, 4); total time= 0.5s
[CV] END clf__alpha=1.0, ngram__min_df=2, ngram__ngram_range=
=(2, 4); total time= 0.4s
[CV] END clf__alpha=1.0, ngram__min_df=2, ngram__ngram_range=
=(2, 4); total time= 0.4s
[CV] END clf__alpha=1.0, ngram__min_df=2, ngram__ngram_range=
=(3, 5); total time= 0.4s
[CV] END clf__alpha=1.0, ngram__min_df=2, ngram__ngram_range=
=(3, 5); total time= 0.4s
[CV] END clf__alpha=1.0, ngram__min_df=2, ngram__ngram_range=
=(2, 4); total time= 0.4s
[CV] END clf__alpha=1.0, ngram__min_df=2, ngram__ngram_range=

```

```

=(3, 5); total time= 0.4s
[CV] END clf__alpha=1.0, ngram__min_df=5, ngram__ngram_range=
=(3, 5); total time= 0.3s
[CV] END clf__alpha=1.0, ngram__min_df=5, ngram__ngram_range=
=(3, 5); total time= 0.4s
[CV] END clf__alpha=1.0, ngram__min_df=5, ngram__ngram_range=
=(3, 5); total time= 0.3s
[CV] END clf__alpha=1.0, ngram__min_df=5, ngram__ngram_range=
=(2, 4); total time= 0.4s
[CV] END clf__alpha=1.0, ngram__min_df=5, ngram__ngram_range=
=(2, 4); total time= 0.4s
[CV] END clf__alpha=1.0, ngram__min_df=5, ngram__ngram_range=
=(2, 4); total time= 0.4s
Best params: {'clf__alpha': 0.1, 'ngram__min_df': 2,
'ngram__ngram_range=': (3, 5)}

```

Best CV score: 0.9322648302379962

Test macro F1: 0.8782

Test micro F1: 0.9387

	precision	recall	f1-score	support
ADJ	0.8998	0.8794	0.8894	2470
NOUN	0.9540	0.9641	0.9590	6861
VERB	0.8417	0.7372	0.7860	137
accuracy			0.9387	9468
macro avg	0.8985	0.8602	0.8782	9468
weighted avg	0.9382	0.9387	0.9384	9468

Saved best overall model to:

output_pos_word_classifier\best_model_linear_svm.joblib

Best overall macro F1: 0.9485